



The SIGIL Failure Atlas

How a Chain and Its Lineage Actually Broke

A companion to *SIGIL — The Variational Chain*, whitepaper v1.3

v0 · 2026-07-23

bitknight.dipper688@passmail.net

SIGIL / Flux Foundation · sigilgraph.quillon.xyz

“No silent corruption serving traffic. Ever.”

— the doctrine this atlas is the evidence for

Scope & method. This atlas catalogues **71** distinct failures observed or deliberately induced across SIGIL, its production sister network Quillon Graph, the Flux toolchain, and the four-node fleet, 2026-05 to 2026-07. Every entry is drawn from the project’s own incident record — memory notebooks, the 2026-06-10 security audit, the hardening backlog, and the production-instability diagnosis — with numbers, heights, and file:line quoted verbatim so each is re-checkable. It is pseudonymous and secret-free: incident *facts* are recorded (“a key was exposed”), never the secrets themselves. Open items are marked **OPEN** and not argued away. Failure here is not confessed; it is *cited*, because in this project a failure is the highest-grade evidence a system produces about itself.

15

CATASTROPHIC — funds,
consensus, or fleet at stake

44

MAJOR — outage, cor-
ruption, or exploit class

12

MODERATE — degraded,
misleading, or latent

1 Why a chain keeps an atlas of its own failures

Most systems bury their failures. The post-mortem is internal, the incident channel is private, and the public artifact — the whitepaper, the launch post — describes the system as designed, not as it broke. This atlas takes the opposite view, and it is the view the rest of the SIGIL corpus already commits to: that a design is a hypothesis, and its failures are the experiments that test it. The whitepaper’s proudest mechanism, the four-state-root header that halts a node on divergence (exit code 78), exists because of a specific dated injury on the production sister network — a replay bug that silently dropped a wallet from 3200 to 1484 QUG with no protocol-layer signal. *The architecture is a scar map.* You cannot read it honestly without the wounds.

So this companion collects the wounds. It is not a security disclosure (that is the whitepaper’s job, §3.7 there) and not a roadmap. It is a *taxonomy*: seventy-one failures sorted into the shapes they keep taking, so that the next one is recognizable before it is catastrophic. Three things make it worth reading rather than wallowing. First, the failures are *real and cited* — every one carries the file, the number, the height it was found at. Second, they *cluster*: ten classes cover all seventy-one, and a handful of deep patterns cover the classes. Third, and most important, the majority produced a *structural guard* — a rule, an invariant, a default — that makes the whole class harder to re-enter. The atlas’s real subject is that conversion: how a system turns its own failures into the shape of its defenses.

IN PLAIN WORDS — a scar map, not a confession

Every serious bridge, aircraft, and operating system is designed around a list of ways earlier ones failed. That list is not an embarrassment; it is the most valuable document the field owns. This paper is SIGIL's version — written down, numbered, and public, so the same mistake has to be a *new* one to happen again.

1.1 The taxonomy

Every failure in the record falls into one of ten classes. They are not equally dangerous, and they are not independent — the worst incidents are usually two classes stacked (a bad default that produces resource exhaustion; a stale state that hides a silent divergence). The classes, by count:

Class	#	What goes wrong
Silent divergence	12	Two honest parts compute different truths and nothing alarms. The class exit 78 exists to kill.
Protocol wedge	10	An ingress path accepts what it should verify — a length-only check, an un-called verifier, a trusted-but-forgable field.
Stale state	9	A cached, lagging, or reverted value is served or trusted as current — old binaries, old DBs, old flags, un-bumped nonces.
Resource exhaustion	8	Memory or disk climbs unbounded; the OOM family.
Bad default	7	A shipped default is wrong for the real environment — a cap below baseline, a terminal check false on real terminals.
Measurement	7	The instrument lies: a microbenchmark, a dashboard, a unit, a green test that tested nothing.
Economic	6	Value is created, doubled, or destroyed — phantom mints, double-spends, decimal drift.
Key management	6	The trust root is wrong: no auth, a key taken from the attacker's own message, a secret in a public repo.
Toolchain	5	The build or crypto substrate is subtly wrong — a poisoned cache, a benchmark modulus in production, a glibc ceiling.
Serialization panic	1	A value no serializer expected crash-loops the node.

The rest of the atlas walks the classes that recur, features the most instructive incident in each as a full entry, and tabulates the others. It closes with the deep patterns beneath the classes (§8), the guards the failures built (§9), and the exhibits still open (§10).

2 Silent divergence — the class the architecture is built against

If SIGIL has an original sin it is designed to abolish, this is it: two compliant nodes, or two code paths in one node, arrive at different truths, and no signal fires. It is the most dangerous class because it is the quietest, and it is why the whitepaper makes agreement a committed, recomputed quantity rather than a social one.

SD-1 · Fresh nodes hold different funds — turbo-sync skips the DEX

2026-07-16 · quillon

CATASTROPHIC OPEN

SYMPTOM	A P2P-synced fresh node and the authoritative node, at the <i>same</i> height (~h20.72M), held different real balances: a 69,028 QUG native gap over the same 6425 wallets (balance roots c2657388 vs a6c8a1ff); the fresh node had 0 dex_/lp_token keys against the authoritative node's 140 / 26.
ROOT CAUSE	The bulk turbo_sync.rs path never applied DEX/token/swap effects — swaps were replayed <i>only</i> on the gossip path (main.rs:13683). Every fresh node bulk-syncs, so every fresh node missed DEX state. Compounded: the balance root hashed only wallet_balance_* keys — it was native-QUG-only, structurally <i>blind</i> to the token/pool divergence it should have caught. The chain was not truly redundant for ~5 months.
DETECTION	A deliberate two-node cross-check (read-only audit tool a6912fa2) — not an alarm. The guard that should have fired was the very root that was blind.
GUARD	Fix direction (not fully shipped): unify bulk / gossip / replay into one idempotent total apply including DEX; a <i>multi-asset</i> balance root; a signed state anchor; N-node soak proving “kill the authoritative node → a fresh node syncs to an identical multi-asset root” before arming a reject gate.
LESSON	Two code paths that both mutate state will diverge unless unified into one idempotent apply — and a state root that hashes only one asset class cannot detect the divergence it exists to guard.

The purest instance of the class, because both halves are textbook: a second apply-path that silently disagreed with the first, *and* a detector engineered to miss it. The whitepaper's four-root header is the general answer — but SD-1 is the reminder that a root only guards what it hashes. Two more from the same class, in brief. The **peerless self-certifying singleton** (2026-06-13, **CATASTROPHIC**): a production node that lost all peers kept producing and self-certifying blocks, bypassing the BFT quorum entirely — a quorum of one is not a quorum. The **Epsilon/Delta silent fork at ~h18.48M** (2026-06-12, narrowed to 18,475,000–18,490,000 by block-hash bisection): the mesh sat fully de-meshed for weeks, every node solo-producing, each certain it was canonical. And the two *aliasing mints* the SIGIL audit found — a native **Send** with **from==to** whose credit re-read pre-debit state and overwrote the debit (net gain = the amount), and **native_supply** computed with **wrapping_** arithmetic so the 21M cap was checked against a sum mod 2^{128} — are silent divergence in miniature, inside a single transaction.

IN PLAIN WORDS — why “it matches” is the hard part

Two calculators agreeing on $2+2$ is easy. Two computers, run by different people, independently agreeing on the exact balance of every wallet after twenty million transactions — and *noticing the instant they don't* — is the whole problem of a blockchain. Most entries in this section are variations of the same discovery: the agreement was never as total as it looked, and the thing meant to notice was looking at the wrong half.

3 Economic failures — value from nothing, and value into nothing

Silent divergence is dangerous in general; when the divergent quantity is *money*, it is the whole game. Six incidents created, doubled, or destroyed value directly.

EC-1 · The \$58.5-trillion phantom swap-mint 2026-07-02 · quillon **CATASTROPHIC** **PARTIALLY GUARDED**

SYMPTOM	A single wallet (4cf3b55e) acquired a \$58.5-trillion QUGUSD position through a swap that minted the output without conserving the input, the reconciler then treating the phantom balance as real.
ROOT CAUSE	The swap-apply path and the balance reconciler did not enforce a conservation invariant across the swap: output credited, input not correspondingly debited, and no apply-layer check that total value in = total value out. One of three open mint vectors (coinbase, CDP, bridge) the 2026-06-24 mint audit catalogued alongside a partial DEX-decimal issue.
GUARD	Primary clawback <i>executed</i> 2026-07-11 (the top abuser wallets swept 58.60T → 3.79B QUGUSD); a round-2 sweep of grey wallets is paused by the operator. The durable fix is still pending: an apply-layer conservation invariant on every state transition (the SIGIL chokepoint's per-delta conservation check is the sister design). The audit's finding — “no apply-layer conservation invariant” — is the general defect.
LESSON	Every money mutation needs a conservation check <i>at the apply layer</i> , not just at the API; a swap that credits without debiting is a mint with extra steps.

EC-2 · The mixer double-spend — max-wins drops the debit 2026-06-27 · quillon CATASTROPHIC

GUARDED

SYMPTOM	Two independent drains. Normal sends did not debit the sender (coins re-spendable forever); the quantum mixer allowed the same coins to be spent twice.
ROOT CAUSE	Vector 1: <code>send_signed</code> built the transaction with an <i>empty</i> signature, relying on an in-memory <code>trusted_via_auth</code> flag that does not propagate over P2P — so miners rejected the unsigned tx, it never landed in a block, and the sender was never debited. Vector 2: the mixer wrote balances directly (no block/consensus) via <code>save_wallet_balances</code> , a <i>batch max-wins</i> write that skips any entry where existing $\geq$ new — so the sender's <i>debit</i> (new < old) was silently dropped while the recipient's credit landed; on the next RocksDB reload the spent coins reappeared. An infinite mint hiding inside a safety rule.
GUARD	Client-side Ed25519 signing (<code>signTransferV72</code> reproduces the signable payload byte-for-byte); unsigned rejected unless an explicit escape flag is set. The mixer now persists <i>both</i> legs with an <i>authoritative</i> force-write (<code>save_wallet_balance_authoritative</code> , no max-wins) and requires an authenticated wallet.
LESSON	A max-wins write protects against stale <i>overwrites</i> and, applied to a legitimate debit, becomes a mint — the safety rule and the exploit were the same line of code, read in two directions.

EC-2 is the atlas's sharpest irony and the reason the sister chain's balance-integrity rules are non-negotiable: `save_wallet_balances` being max-wins is *correct* — it is exactly how a replay bug once dropped a wallet 3200 → 1484 QUG and how the project swore never again — and it is precisely that correctness that ate the mixer's debit. The lesson is not “max-wins is wrong”; it is that a debit is not an overwrite, and a persistence layer that cannot tell them apart will protect the wrong one. The class also holds the **gossip transaction-replay drain** (2026-07-02, CATASTROPHIC: a re-broadcast tx re-applied against the sender), **CULTURE zeroed by a decimal swap** (2026-06-28: a decimal-mismatch debit drove a token's balance store to 0 while its ledger stayed intact — value destroyed, not stolen), and the **Beta 29.6-billion re-fork** (§6), where a backup node minted **29,676,551,397** QUG — 1,413× the 21M cap — on an abandoned fork.

4 Trust roots — authenticating against the attacker's own message

The protocol-wedge and key-management classes share one root confusion: mistaking *integrity* (this wasn't corrupted in transit) for *authenticity* (this came from who it claims to). Ten-plus incidents are variants. The SIGIL money daemon supplies the cleanest set, because they were found by audit before an attacker found them.

TR-1 · A public money daemon with zero authentication 2026-06-10 · sigil CATASTROPHIC GUARDED

SYMPTOM	sigil-rpcd was exposed on public :8099 with <i>no</i> owner-signature check on any money route. The headline: <code>/mine</code> took a <i>caller-supplied</i> difficulty (default 0 → all PoW passes) and reward, so one anonymous <code>curl</code> could mint up to the 21M cap to any wallet (<code>sigil-rpcd.rs:777-793</code>). <code>/swap</code> and <code>/add_liquidity</code> spent any from wallet with no signature; <code>/onboard{}</code> was an unlimited +100/+1000 faucet; <code>/deploy_token</code> minted any uncapped token.
ROOT CAUSE	sigil-oauth — real OAuth2/PKCE, Ed25519/SQIsign tokens, fails-closed on unknown algorithms — was <i>fully built and never wired in</i> . The router gated nothing; CORS * on every response compounded it.
GUARD	A wallet-signature gate (Ed25519 over a domain-separated canonical message with a per-wallet replay-nonce persisted in the snapshot) wired into every money route; <code>/mine</code> now derives reward and difficulty server-side. Deployed <i>enforcing</i> 2026-06-10, verified (unsigned <code>/mine /swap /credit /deploy_token</code> → “missing sig”); 42/42 crate tests.
LESSON	A correct auth library that is never wired into the request path provides exactly zero protection; the whole fix was integration, not cryptography.

TR-2 · The updater that verified releases against the attacker’s key2026-06-10 · sigil CATASTROPHIC GUARDED

SYMPTOM	The auto-updater verified a release’s signature against <code>a.sqisign_pubkey</code> — a field of the <i>announcement itself</i> . No trusted-key allowlist existed, so anyone publishing on the release topic could push a self-signed malicious binary: fleet-wide remote code execution.
ROOT CAUSE	Signature verification trusted the public key carried <i>inside the same message being verified</i> — a valid signature over attacker-chosen bytes, with no root of trust anywhere.
GUARD	<code>verify_announcement_pinned</code> : a pinned constant key plus an out-of-band allowlist; empty allowlist fails closed (auto-update off). Announcements whose key isn’t allowlisted are rejected <i>before</i> verification. 6/6 tests.
LESSON	Verifying a signature against a key supplied by the same object authenticates nothing — the trust anchor must be pinned out of band, or the “signature” is decoration.

TR-1 and TR-2 are the same mistake at two layers, and the same mistake as the wire-level **keyless tip-proofs** (H4, **OPEN**): tip-proofs verified only by a keyless BLAKE3 fingerprint whose own doc-comment reads “any peer can fabricate one,” while the real SQIsign verifier exists but is never called on the wire. Integrity, mistaken for authenticity, three times. The rest of the class is the SIGIL hardening backlog’s consensus ingress: **block-apply verifies no producer signature** (H1, **CATASTROPHIC**, **PARTIALLY GUARDED** — a length-only precheck with `producer_sig = vec![0u8;292]`; the height-gated verifier is written, tested, and *dormant* at activation height `u64::MAX`); the **length-only sync handshake** (H2 — verification checked `identity_sig.len()==32`, and nothing called it); the **follower that trusts peer-declared difficulty** (H3 — serve `bits=4`, the follower credits the full reward); and the **bridge mint unbound from its proof** (C9, **CATASTROPHIC** — an SPV proof established only that *some* transaction existed while amount and recipient were caller-chosen, with no spent-set, so one proof mints unlimited wrapped assets N times).

IN PLAIN WORDS — a wax seal proves the letter is sealed, not who sealed it

Half the worst bugs in this atlas are one confusion: checking that a message is intact is not the same as checking who it is from. A seal nobody can forge is worthless if the seal-maker is whoever sent the letter. The fix is always the same shape — decide *in advance*, out of band, whose seals you trust, and reject everyone else before you even look.

5 Resource exhaustion — the OOM family

Eight incidents are memory or disk climbing until something dies. They rarely corrupt state, but they take the node — and on a single-producer network, the node is the chain.

RX-1 · The unbounded prefix-scan OOM death-loop2026-06-23 · quillon MAJOR GUARDED

SYMPTOM	The production node climbed 0 → ~47 GB anon-RSS in 30–90 s , hit the cgroup MemoryMax (46 G, bumped to 48 G), took status=9/KILL (“Memory cgroup out of memory”), ~1 kill every 2.5 minutes, restart count in the hundreds; users saw intermittent ERR_CONNECTION_RESET / 502.
ROOT CAUSE	An unbounded RocksDB prefix scan materialized an entire key class into memory on a hot path — the working set exceeded any cap because it grew with the chain, not with the request.
GUARD	Bound the scan; the incident also hardened the OOM posture (below). Diagnosis lives in the production-instability writeup.
LESSON	Any scan whose size grows with chain length, not request size, is an OOM waiting for the chain to get long enough.

The class’s most instructive member is not the biggest but the most *misdiagnosed*: **jemalloc retention read as a leak** (2026-06-13) — RSS that looked like an unbounded leak was the allocator holding freed pages, fixed with a `_RJEM_MALLOC_CONF` background-thread + decay env setting and *no code change at all*. Chasing it as a leak would have burned days on the wrong layer. The others: the **Bracha-consensus instance leak** (2026-06-19, a 48h OOM freeze from consensus instances never reclaimed); the **mining-challenge × LWMA stampede** (2026-06-12, challenge regeneration triggering full-block fetches in a herd); the **producer $O(N^2)$ snapshot decay** (2026-06-08, a $1,075\times$ OOM loop as snapshot cost grew quadratically); and — the one that crossed chains — **the release ceremony that filled the disk** (2026-07-21): a SIGIL rollout filled `/home` to 100%, which crash-looped the *Quillon* production node sharing the box until 250 G was freed. A neighbor’s celebration is a resource attack.

6 Bad defaults and stale state — the config that was wrong for the world

Sixteen incidents (7 bad-default, 9 stale-state) share a shape: the code was arguably correct in the abstract and wrong for the actual environment, or served a value that used to be true. The single most expensive entry in the whole atlas is here.

BD-1 · The 1-GiB mesh wedge — an OOM cap below the baseline

2026-06-24 · quillon

CATASTROPHIC GUARDED

SYMPTOM	A months-long fleet de-mesh. The authoritative node’s peer list was empty (0 libp2p peers), new nodes could not join, dialers saw UnexpectedEof / BrokenPipe right after the Noise handshake. Every node holding the 219 GB chain refused every other node.
ROOT CAUSE	<code>libp2p:memory_connection_limits</code> was wired with a 1 GiB default cap (<code>unified_network_manager.rs:2128</code>), env unset on the fleet. That behaviour refuses <i>every</i> new connection — inbound and outbound — with <code>ConnectionDenied{MemoryUsageLimitExceeded}</code> whenever process RSS exceeds the cap. Real steady-state RSS was 5.5 GB . The OOM-defence threshold sat <i>below every real node’s normal working set</i> , so it silently became a total connectivity block.
DETECTION	A controlled 2-node Docker experiment: a 1 MiB cap → 0 connections (dialer times out); raise it → ConnectionEstablished immediately. The wedge was invisible to every log because refusal was the designed behaviour.
GUARD	Set the cap to 40 GiB fleet-wide (above baseline, below the 38 G MemoryMax); operating rule <code>Q_P2P_ONLY=1</code> with no HTTP sync fallback (HTTP only discovers height, transfers no blocks). A fresh Docker node then pulled real mainnet block-packs over the public interface.
LESSON	A safety threshold set below a process’s normal working set does not degrade gracefully — it converts, silently, into a total denial of the thing it guards. Defaults must be validated against measured steady state.

BD-1 is the atlas’s paradigm case for “measure the live path first,” the sister rule of the philosophy companion: the cap was a *reasonable-sounding number* (1 GiB of connection memory) that had never been checked against the 5.5 GB reality, and it cost the network its mesh for months while every node logged nothing wrong. The rest of the stale/default family is a catalogue of environment mismatches: the **`is_terminal()` check false on real Windows consoles** (2026-06-19 — so the TUI never started; the fix was to *always* run it), the **hardcoded `/root` DB path** that filled a 40 GB root partition when a relative `Q_DB_PATH` resolved off the big disk, the **bootstrap peer registered under its own peer-id** (2026-06-09, so the DHT never seeded and the mesh sat at 0 peers), the **two-target-dirs stale-binary**

publish (2026-06-10, shipping an old binary from the wrong build dir), and **the stale global connlimit rule** that took **quillon.xyz** down (2026-06-20, an 80-connection cap on `:443` left over from an unrelated change). Each was correct somewhere; none was correct here.

The class's most vivid stale-state incident is the **Beta 29.6-billion re-fork** (2026-06-30, **MAJOR**): a backup node re-diverged onto an abandoned phantom-mint fork and reported a supply of **29,676,551,397** QUG — **1,413×** the 21M cap — at height 19,478,848 (~82k blocks behind). Users briefly saw their wallet flicker through impossible balances (8M, 1B) because the staging subdomain **beta.quillon.xyz** was wired *directly* to that node. The structural guard was already in place and held: the apex **quillon.xyz** reads the authoritative node only (failover to backups deliberately disabled 2026-06-11), so the forked values never reached production — the fix was to stop Beta, keep its fork DB for forensics, and resync a fresh DB from the authoritative chain. A staging endpoint wired straight to a node can leak a corrupt fork's balances; production endpoints must stay pinned to the authoritative node.

7 Measurement failures — when the instrument lies

Seven incidents are the instrument itself being wrong — and they are the class the project fears most, because a lying instrument corrupts every decision made from it. The paradigm gave the whole corpus its Rule 0.

MS-1 · The five-day 800-blk/s stall behind a 10-million-blk/s bench

2026-07 · sigil **MAJOR**

GUARDED

SYMPTOM	Live block-sync sat at ~800 blocks/s for <i>five days</i> while the local commit pipeline microbenchmarked at 10,000,000 blocks/s durable and an in-process end-to-end harness reached 39,112 blk/s . Enormous effort went into the fast pipeline; the live number never moved.
ROOT CAUSE	The real caps were elsewhere entirely: a producer-side full-block serve throttle (≤ 1 per 120 ms ≈ 8 serves/s) and a serial request→await→apply fetch loop (one range in flight). The local pipeline was never the bottleneck; the microbenchmark next to the live number <i>was the bug report</i> .
GUARD	Removed the throttle (finalized ranges cache-served by memcpy) and added a request-ahead window; the live number moved 2,322 → 10,853 blocks/s . The episode became Rule 0 of the SIGIL skill: measure the live path first; a fast microbench beside a slow live number is a bug in your understanding, not a victory.
LESSON	Optimizing a component you have not proven is the bottleneck is motion, not progress — and a benchmark that disagrees with production is measuring the wrong system.

The class's other members are all instruments that quietly lied. The **0/0 green test trap** (2026-07-19): a test harness reported **0 passed / 0 failed** as success while a stale server ran a deleted executable — zero tests run reads identically to zero tests failed unless you gate on the verdict, not the failure count. The **lying dashboards** (2026-06-13): roughly 34 fixes chased against telemetry that was itself broken — `_ms` fields carrying unix *seconds*, metrics never seeded, phantom version numbers — producing fix-then-regression cycles until someone audited the instruments instead of the system. And, one class over but the same disease, the **delivery-law regrade** that this atlas's sibling documents celebrate: a law graded *measured* whose validating simulator shared the law's own independence assumption — caught, regraded to derived+sim, and only promoted back to measured after a real netem experiment. The instrument was believed a little too early; the discipline caught it.

8 The patterns beneath the classes

Ten classes is still too many to hold in the head at the moment of writing code. Beneath them are a handful of deep shapes that generate most of the catalogue. These are the atlas's real payload — the things to check for before the class has a chance to form.

1. **Built-but-not-wired.** The dominant shape. A correct primitive exists while the live path keeps calling the insecure or absent one. The zero-auth daemon (TR-1, **sigil-oauth** built, unwired), the forgeable VDF modulus (C10, trustless **rsa2048()** built, live callers still on **bench_2048**), the replay nonce (H5, **check_and_bump_nonce** committed with *zero callers*), the dormant producer-sig verifier (H1), the never-called SQIsign tip-proof (H4). *The vulnerability was integration, never cryptography.* A fix that lands on a branch and never flips its activation switch is not a fix; it is a fix-shaped object.

2. **Length-as-verification.** Type-checking a field’s *size* mistaken for validating its *contents*: the length-only block precheck with a zeroed signature (H1), the handshake that checked `identity_sig.len()==32` (H2). If your “verification” would pass on a field of the right length filled with zeros, it verifies nothing.
3. **Integrity mistaken for authenticity.** The trust-root confusion of §4: a key taken from the signed message (TR-2), a keyless fingerprint used as a trust anchor (H4). Intactness is not provenance.
4. **Caller-chosen where proof-derived is required.** A consensus-critical value trusted from the request instead of derived from proven state: `/mine` difficulty and reward (TR-1), swap `from` (TR-1), bridge amount/recipient (C9), follower- trusted difficulty bits (H3), self-declared SPV difficulty (H7). The recurring fix is one sentence: *derive it, don’t accept it*.
5. **Two paths, one state.** Any state reachable by two code paths will diverge unless the paths are one idempotent apply: turbo-sync vs gossip (SD-1), mixer-direct-write vs consensus (EC-2), bulk vs replay balances. The whitepaper’s single `commit_state_transition` chokepoint is the structural answer — one door, or two truths.
6. **The safety rule that became the exploit.** Max-wins persistence protecting against stale overwrites *and* eating a legitimate debit (EC-2); an OOM cap protecting memory *and* refusing all peers (BD-1). The most dangerous line is the one that is correct in the case you designed it for and catastrophic in the case you didn’t.
7. **The audit decays.** By the 2026-07-18 ground-truthing, one audit finding (H6) was already fixed and several (C9, C10, H5) were further along than the 2026-06-10 audit claimed. A stale audit is itself a lying instrument (§7). The house rule that fell out: *when the audit and the code disagree, read the code*.

IN PLAIN WORDS — seven things to check before you ship

Almost every serious failure in this paper is one of seven questions someone didn’t ask: Is the secure version actually the one that runs? Am I checking a thing’s size when I mean its contents? Am I confusing “unbroken” with “genuine”? Am I trusting a number the sender chose? Can this data be written two ways? Is my safety rule safe in the case I *didn’t* imagine? And — is the report I’m trusting still true? The atlas exists so these become reflexes.

9 What the failures built

The atlas would be a misery if it ended at the catalogue. It does not, because most of these classes produced a *structural* guard — not a patch, but a change to the shape of the system that makes the class harder to re-enter. This is the conversion the paper is really about.

The failure	The structural guard it built
Silent balance divergence (the founding 3200→1484 injury)	Four committed state roots, independently recomputed, self-halt on mismatch (exit 78) — agreement made checkable, not social.
turbo-sync skipped the DEX (SD-1)	The target invariant: one idempotent total apply, a <i>multi-asset</i> root.
Max-wins ate the mixer debit (EC-2)	The Balance Integrity Rules: max-wins for overwrites, authoritative force-write for real debits — the two are never the same path.
Zero-auth money daemon (TR-1)	Every money route fails closed on a per-request wallet signature with a replay nonce; server derives consensus-critical values.
Updater RCE from in-message key (TR-2)	Out-of-band pinned release keys, fail-closed on empty allowlist.
The 1 GiB mesh wedge (BD-1)	“Measure the live path first” as an operating rule; caps validated against measured steady-state RSS.
The five-day stall (MS-1)	Rule 0: a microbench beside a slow live number is a bug report; exactly one owner of the real end-to-end number.
OOM death-loops (RX-*)	OOM-guard posture: negative <code>OOMScoreAdjust</code> on prod, capped <code>systemd-run</code> scopes for test workloads, allocator decay tuning.
The audit decayed (§8)	“When the audit and the code disagree, read the code” — findings ground-truthed against source before work.

The pattern of the pattern: a silent failure produces a *loud* guard (exit 78, a fired alarm, a fail-closed default), and a measurement failure produces a *methodological* guard (a rule about how to measure). The system does not merely patch; it raises its own floor. That is the only honest reason to keep an atlas — not to remember the wounds, but to keep the scar tissue load-bearing.

10 Still open — the exhibits without a guard yet

Honesty requires ending on the failures that are *not* yet closed, graded as such, per the house rule that open items are marked and not argued away. These are live or dormant as of 2026-07-23.

Open exhibit	Status	What closes it
H1 — block-apply verifies no producer signature	PARTIALLY GUARDED dormant	Producer-side signing ships; hot path to a fast scheme or validator registry; flip the activation height off <code>u64::MAX</code> after Alpha soak.
C10 — VDF over a forgeable benchmark modulus	PARTIALLY GUARDED	Wire the trustless <code>rsa2048()</code> into <i>every</i> verify + mining call site, height-gated with a coordinated miner upgrade.
H4 — keyless wire tip-proofs	OPEN	Produce <i>and</i> verify <code>SQIsign</code> tip-proofs on the wire; keep <code>BLAKE3</code> integrity-only, never a trust root.
H5 — replay nonce committed but uncalled	PARTIALLY GUARDED	Route the live apply through the signed-batch path so the nonce store actually gates.
SD-1 — fresh nodes miss DEX state; native-only root	OPEN	One idempotent total apply including DEX; multi-asset root; signed anchor; N-node soak before arming the reject gate.
EC-1 — phantom swap-mint; no apply-layer conservation	PARTIALLY GUARDED	Primary clawback done (2026-07-11); still needs the apply-layer conservation invariant on every transition (and the paused round-2 sweep).
/onboard unlimited faucet mint	OPEN	Finite faucet-debit accounting + per-IP rate limit + decouple the state-persist from the call path.

Note the shape of the open list: it is dominated by *built-but-not-wired* and *caller-chosen* — the two deepest patterns of §8, which is exactly why they are the last to close. The guards are designed; the

switches are not yet flipped. On a single-producer experimental network that is a tolerable posture, stated plainly in the whitepaper's own security disclosure: do not route significant value through SIGIL until they close. This atlas is the itemized version of that sentence.

v0 — 2026-07-23. A design is a hypothesis; its failures are the experiments. The scar map is the architecture. Seventy-one wounds, ten shapes, seven questions — and a system that turns each closed one into a floor it can no longer fall below.

Sources

- *SIGIL — The Variational Chain*, whitepaper v1.3 (2026-07-22) — the design this atlas is the shadow of; https://quillon.xyz/downloads/SIGIL_WHITEPAPER_v1.pdf
- *What the Chain Knows — The Philosophy of the SIGIL Graph*, v0.3 (2026-07-22) — the epistemology of honesty this atlas practices; https://quillon.xyz/downloads/SIGIL_PHILOSOPHY_v0.pdf
- *The Builder's Chronicle*, v0 (2026-07-23) — the fourth companion: how the chain was built, development settled on-chain as a prototype labor institution; https://quillon.xyz/downloads/SIGIL_BUILDERS_CHRONICLE_v0.pdf
- *The Adversary's Companion*, v0 (2026-07-24) — the fifth companion: this atlas turned into a threat model, the open items read as graded attack trees from the attacker's chair; https://quillon.xyz/downloads/SIGIL_ADVERSARYS_COMPANION_v0.pdf
- *SIGIL Security Audit* (2026-06-10) and *SIGIL Hardening Backlog* (2026-07-18) — the C1–C10 / H1–H8 findings and their verdicts (in-tree).
- *Quillon Production-Instability Diagnosis* (MASTER-DIAGNOSIS + addenda, 2026-06-13/17) — the death-loop, singleton, silent-fork, and telemetry findings.
- The project incident memory (2026-05 to 2026-07) — the fleet, fork, mesh, OOM, double-spend, and toolchain records this atlas is drawn from and verified against.